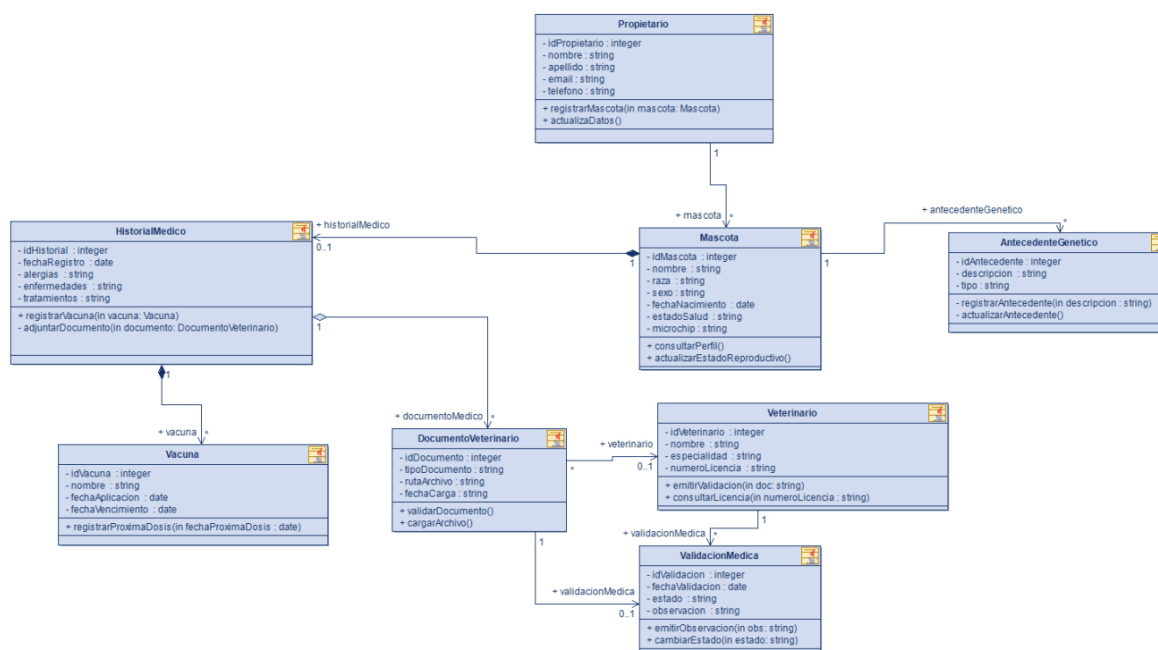


4 MODELO UML DE ORIGEN

4.1 Diagrama y clases del subsistema

El modelo UML del subsistema de gestión de información de mascotas y control veterinario fue desarrollado en Modelo 5.4.01. El diagrama de clases constituye el modelo de origen utilizado por Java Designer 5.4.00 para generar la estructura del código Java.

El modelo contiene ocho clases: Propietario, Mascota, HistorialMedico, Vacuna, DocumentoVeterinario, Veterinario, ValidacionMedica y AntecedenteGenetico. Todas poseen atributos tipados, operaciones, visibilidad y relaciones con multiplicidades definidas.



Las responsabilidades principales se resumen en la Tabla

Clase	Responsabilidad
Propietario	Registrar y administrar las mascotas de su perfil.
Mascota	Centralizar los datos generales y sanitarios del animal.
HistorialMedico	Organizar enfermedades, tratamientos y registros médicos.
Vacuna	Registrar vacunas y sus fechas de aplicación y vencimiento.
DocumentoVeterinario	Representar certificados, informes y archivos médicos.
Veterinario	Consultar información y emitir validaciones profesionales.
ValidacionMedica	Registrar el estado y las observaciones de una validación.
AntecedenteGenetico	Registrar antecedentes y riesgos hereditarios.

4.2 Relaciones del modelo

El diagrama incorpora asociaciones, agregaciones y composiciones. Las multiplicidades indican la cantidad de objetos que participan en cada relación y permiten que Java Designer genere referencias simples o colecciones tipadas.

- **Propietario–Mascota:** asociación uno a muchos. Cada mascota pertenece a un propietario (1) y un propietario puede administrar cero o varias mascotas (0..*).
- **Mascota–HistorialMedico:** composición. Una mascota puede tener un historial (0..1) y cada historial pertenece a una mascota (1).
- **Mascota–AntecedenteGenetico:** asociación. Una mascota puede registrar varios antecedentes (0..*) y cada antecedente pertenece a una mascota (1).
- **HistorialMedico–Vacuna:** composición. Un historial puede contener varias vacunas (0..*) y cada vacuna pertenece a un historial (1).
- **HistorialMedico–DocumentoVeterinario:** agregación. Un historial puede reunir varios documentos (0..*) y cada documento puede asociarse con un historial (0..1).
- **Veterinario–DocumentoVeterinario:** asociación. Un veterinario puede revisar varios documentos (0..*) y cada documento puede relacionarse con un veterinario (0..1).
- **Veterinario–ValidacionMedica:** asociación. Un veterinario puede emitir varias validaciones (0..*) y cada validación corresponde a un veterinario (1).
- **DocumentoVeterinario–ValidacionMedica:** asociación. Un documento puede tener una validación (0..1) y cada validación se relaciona con un documento (1).

Las relaciones múltiples pueden transformarse en colecciones como List, List o List. El modelo no incorpora generalización, debido a que Propietario y Veterinario se representan como clases independientes.

4.3 Estereotipo y validación

Para habilitar la generación se utilizó el perfil de Java Designer 5.4.00 y se aplicó a las ocho clases el estereotipo:

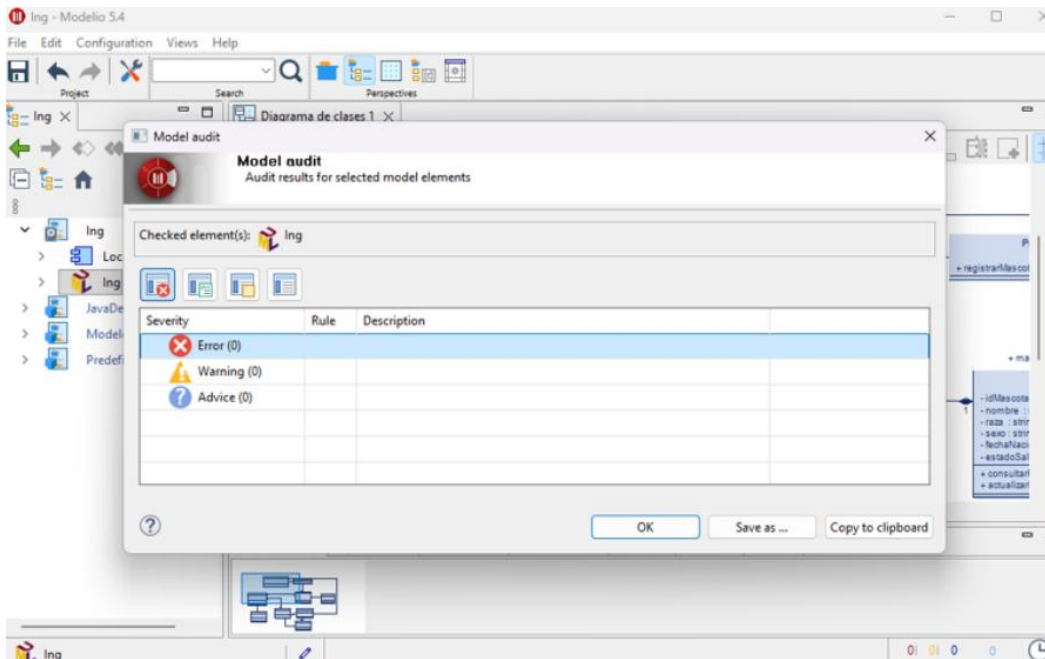
«JavaClass»

No se emplearon los estereotipos «Entity», «Service» ni «Controller», porque el generador produce estructuras Java y no implementa automáticamente persistencia, servicios o una arquitectura por capas.

La validación se realizó seleccionando el paquete principal y ejecutando:

Audit → Check Model

El resultado registró cero errores, cero advertencias y cero recomendaciones.



La auditoría se complementó con una revisión manual de tipos, firmas de operaciones, multiplicidades, nombres de rol y estereotipos, debido a que algunos problemas solo se manifiestan durante la generación o compilación del código.

4.4 Archivo nativo del modelo

El proyecto nativo de Modelio, el diagrama UML y las evidencias se encuentran almacenados en el repositorio público:

<https://github.com/MoralesGaryUteq/PFC-MundiPets-MDD-EquipoE>

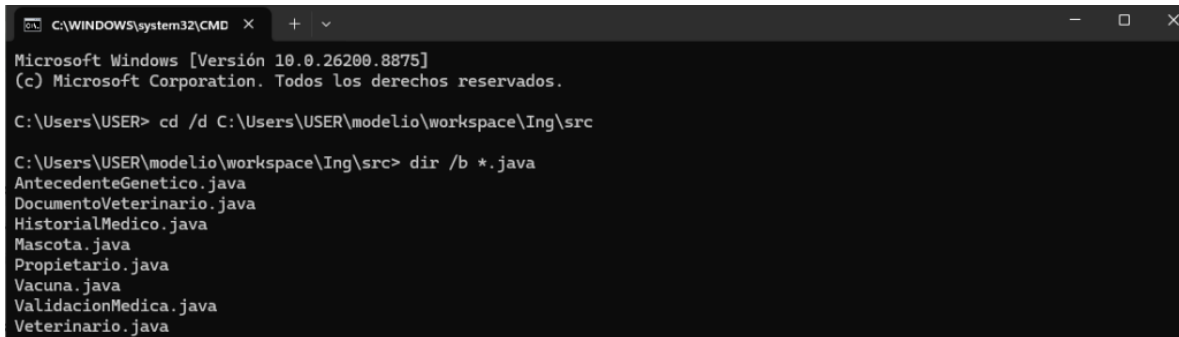
6 VERIFICACIÓN DEL SOFTWARE GENERADO

6.1 Proceso de compilación

La verificación se realizó desde la consola de Windows en el directorio donde Java Designer almacenó el código generado (C:/Users/USER/modelio/workspace/Ing/src), confirmando con `javac -version` la disponibilidad de `javac 17.0.19`. Este compilador no intervino en la generación, realizada por Java Designer 5.4.00; se utilizó únicamente para verificar la validez sintáctica de los archivos obtenidos.

La compilación inicial mediante `javac *.java` permitió detectar parámetros sin nombre, tipos incorrectos y extremos de asociación incompletos, incluida una declaración sin nombre de campo (`private Propietario ;`). Las inconsistencias se corrigieron directamente en el modelo UML y las clases afectadas fueron regeneradas, manteniendo el modelo como fuente principal del código. Tras las correcciones, `javac *.java` no reportó errores y produjo ocho archivos compilados: `AntecedenteGenetico.class`, `DocumentoVeterinario.class`,

HistorialMedico.class, Mascota.class, Propietario.class, Vacuna.class, ValidacionMedica.class y Veterinario.class.



```
C:\WINDOWS\system32\CMD
Microsoft Windows [Versión 10.0.26200.8875]
(c) Microsoft Corporation. Todos los derechos reservados.

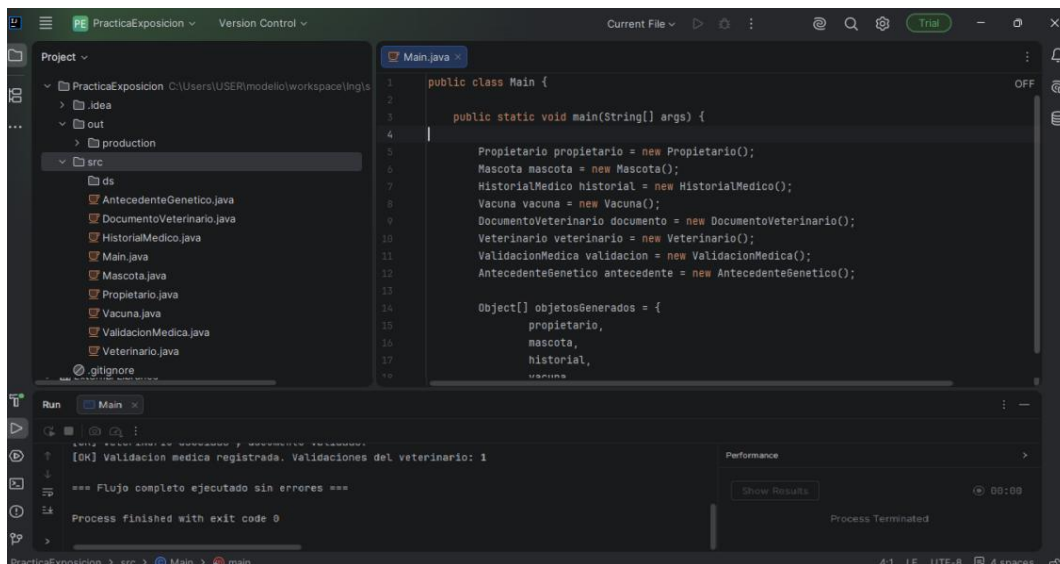
C:\Users\USER> cd /d C:\Users\USER\modelio\workspace\Ing\src

C:\Users\USER\modelio\workspace\Ing\src> dir /b *.java
AntecedenteGenetico.java
DocumentoVeterinario.java
HistorialMedico.java
Mascota.java
Propietario.java
Vacuna.java
ValidacionMedica.java
Veterinario.java
```

La evidencia confirma la correspondencia entre las ocho clases UML, los ocho archivos .java y los ocho archivos .class obtenidos.

6.2 Ejecución de una unidad demostrable

Las clases generadas representan la estructura del dominio, pero no incluyen un método principal, por lo que se creó manualmente la clase Main.java, separada del código producido por Java Designer. Esta unidad instanció las ocho clases (Propietario, Mascota, HistorialMedico, Vacuna, DocumentoVeterinario, Veterinario, ValidacionMedica y AntecedenteGenetico), confirmó el mensaje Total de clases verificadas: 8 y terminó con Process finished with exit code 0.



```
public class Main {
    public static void main(String[] args) {
        Propietario propietario = new Propietario();
        Mascota mascota = new Mascota();
        HistorialMedico historial = new HistorialMedico();
        Vacuna vacuna = new Vacuna();
        DocumentoVeterinario documento = new DocumentoVeterinario();
        Veterinario veterinario = new Veterinario();
        ValidacionMedica validacion = new ValidacionMedica();
        AntecedenteGenetico antecedente = new AntecedenteGenetico();

        Object[] objetosGenerados = {
            propietario,
            mascota,
            historial,
            vacuna,
            documento,
            veterinario,
            validacion,
            antecedente
        };
    }
}
```

Run: Main

[OK] Validacion medica registrada. Validaciones del veterinario: 1

=== Flujo completo ejecutado sin errores ===

Process finished with exit code 0

Performance: 00:00

Show Results

Process Terminated

La clase Main.java corresponde a código manual y no se contabiliza dentro de las ocho clases generadas.

6.3 Resultados y correspondencia modelo-código

La verificación no se limitó a contar los archivos generados. También se compararon las clases, atributos, operaciones y asociaciones definidos en el modelo UML con las declaraciones producidas por Java Designer.

La Tabla presenta la trazabilidad entre elementos concretos del modelo y su representación en Java.

Clase / archivo	Elemento definido en UML	Representación verificada en Java
Propietario	idPropietario:integer; operación registrarMascota(Mascota); asociación mascotas[0..*]	private int idPropietario; método con parámetro Mascota; colección Listmascotas en Propietario.java.
Mascota	idMascota:integer, fechaNacimiento:date y microchip:string	Campos int idMascota, Date fechaNacimiento y String microchip en Mascota.java.
HistorialMedico	fechaRegistro:date; operaciones registrarVacuna(Vacuna) y adjuntarDocumento(DocumentoVeterinario)	Campo Date fechaRegistro y métodos con parámetros Vacuna y DocumentoVeterinario en HistorialMedico.java.
Vacuna	idVacuna:integer, fechaAplicacion:date y fechaVencimiento:date	Campos int idVacuna, Date fechaAplicacion y Date fechaVencimiento en Vacuna.java.
DocumentoVeterinario	idDocumento:integer, rutaArchivo:string; operaciones validarDocumento() y cargarArchivo()	Campos int idDocumento, String rutaArchivo y métodos correspondientes en DocumentoVeterinario.java.
Veterinario	numeroLicencia:string; asociación múltiple con ValidacionMedica	Campo String numeroLicencia y colección List en Veterinario.java.
ValidacionMedica	idValidacion:integer, estado:string; operaciones emitirObservacion(String) y cambiarEstado(String)	Campos int idValidacion, String estado y métodos con parámetros String en ValidacionMedica.java.
AntecedenteGenetico	idAntecedente:integer, descripcion:string y tipo:string	Campos int idAntecedente, String descripcion y String tipo en

		AntecedenteGenetico.java.
--	--	---------------------------

La comparación confirmó una correspondencia verificable entre las ocho clases del modelo, sus atributos tipados, sus operaciones y los ocho archivos Java generados. Las asociaciones con multiplicidad múltiple se representaron mediante colecciones tipadas, mientras que las multiplicidades máximas de uno produjeron referencias simples.

6.4 Limitaciones encontradas

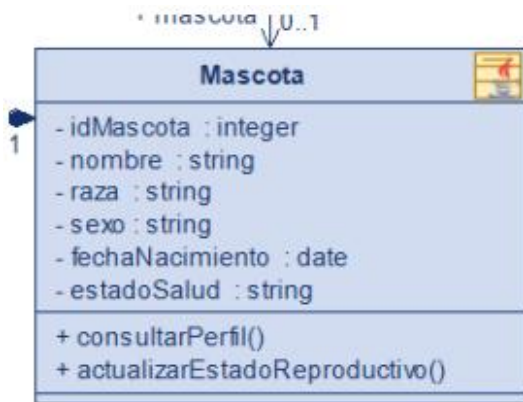
- Java Designer genera principalmente la estructura de las clases, pero no implementa completamente la lógica de negocio.
- La ejecución requiere una clase principal escrita manualmente.
- La persistencia, la interfaz gráfica y la conexión con bases de datos deben incorporarse posteriormente.
- La auditoría automática debe complementarse con revisión manual, compilación y ejecución del código.

En consecuencia, Java Designer facilita la trazabilidad estructural entre UML y Java, pero no sustituye la implementación manual necesaria para obtener una aplicación completamente funcional.

7 ROUNDTRIP ENGINEERING

7.1 Modificación controlada del modelo UML

Para comprobar el cierre controlado del ciclo modelo–código, se incorporó a la clase Mascota el atributo `microchip : string` y la operación `actualizarMicrochip(in nuevoMicrochip:string):void`, que permiten almacenar y actualizar el identificador electrónico de la mascota. Antes del cambio, la clase contenía únicamente los atributos `idMascota`, `nombre`, `raza`, `sexo`, `fechaNacimiento` y `estadoSalud`, y las operaciones `consultarPerfil()` y `actualizarEstadoReproductivo()`.



(a) Clase Mascota antes del cambio.



(b) Clase Mascota después del cambio.

La modificación se considera un cambio estructural compuesto porque afecta simultáneamente un atributo, una operación, un parámetro, un tipo de retorno y la estructura del archivo Java generado. Los elementos se incorporaron directamente en Modelio, manteniendo el modelo UML como fuente principal del código.

7.2 Regeneración del código fuente

Después de modificar la clase Mascota, se ejecutó nuevamente Java Designer → Generate sobre el archivo Mascota.java almacenado en C:/Users/USER/modelio/workspace/Ing/ src. El atributo UML se transformó en el campo private String microchip; y la operación se transformó en el método:

```
public void actualizarMicrochip( final String nuevoMicrochip) { }
```

El cuerpo del método permaneció vacío porque Java Designer genera principalmente la estructura de las operaciones; ni el campo ni la firma fueron escritos manualmente. Tras la regeneración, el código se compiló nuevamente mediante javac *.java sin errores, confirmando que el nuevo atributo, operación y parámetro fueron transformados en estructuras Java sintácticamente válidas.

7.3 Comparación antes y después

La comparación de Mascota.java confirma que los cambios del modelo UML se trasladaron al código fuente sin intervención manual.

(a) Código antes de la regeneración.

(b) Código después de la regeneración.

La Tabla presenta la trazabilidad del cambio realizado.

Elemento	Antes del cambio	Después del cambio
Atributo UML	No existía el atributo microchip.	Se incorporó microchip:string.
Operación UML	No existía una operación para actualizar el identificador.	Se incorporó actualizarMicrochip(in nuevoMicrochip:string):void.
Campo Java	No existía un campo para almacenar el microchip.	Se generó private String microchip;.
Método Java	No existía un método de actualización.	Se generó actualizarMicrochip(String nuevoMicrochip).
Parámetro	No existía un parámetro relacionado con el microchip.	Se generó el parámetro final String nuevoMicrochip.
Intervención manual	No se modificó directamente el archivo generado.	El cambio se realizó en Modelio y se regeneró Mascota.java.
Compilación	El código anterior compilaba correctamente.	El código regenerado volvió a compilar sin errores.

La evidencia demuestra la trazabilidad atributo y operación UML → campo, método y parámetro Java: un cierre controlado del ciclo modelo–código, en el que la modificación se realizó en Modelio y los nuevos elementos aparecieron en Mascota.java sin ser incorporados manualmente.

7.4 Manejo del código manual

La evidencia presentada no constituye una sincronización bidireccional completa, ya que el código Java no fue modificado para actualizar posteriormente el modelo UML. En la configuración estándar de Java Designer 5.4.00 no se comprobó la conservación automática de bloques protegidos, por lo que las ocho clases producidas se trataron como archivos generados y reemplazables. La clase Main.java y cualquier lógica manual se mantuvieron separadas del código generado, evitando que una regeneración sobrescriba la implementación manual.